

# Notas del expositor

## Ataques de envenenamiento de conocimiento a sistemas KG-RAG

Ingrid Alicia Yábar Geldres

Reproducción de Zhao et al. (2025), arXiv:2507.08862

### Resumen

Este documento acompaña a `presentacion.tex` (23 diapositivas) con el guion hablado de cada una, incluyendo el detalle completo de las ecuaciones 4–6 que se fusionaron en una sola diapositiva. Los tiempos sugeridos suman ~**20 minutos** de contenido, dentro de la ventana de 15–20 minutos asignada; la diapositiva marcada como recortable puede omitirse si el tiempo aprieta. Las dos diapositivas de apéndice no se presentan en el flujo normal: son material de respaldo para preguntas puntuales.

## Índice

|                                                                                  |   |
|----------------------------------------------------------------------------------|---|
| 1. Portada                                                                       | 1 |
| 2. Contenido                                                                     | 1 |
| 3. Nota sobre la disponibilidad de código                                        | 1 |
| 4. ¿Por qué importa la seguridad de KG-RAG?                                      | 2 |
| 5. Ejemplo motivador                                                             | 2 |
| 6. Grafo de conocimiento y caminos de relaciones                                 | 2 |
| 7. Pipeline de un sistema KG-RAG                                                 | 2 |
| 8. Modelo de amenaza (caja negra)                                                | 3 |
| 9. Formalización: objetivo de extracción de caminos (I–II) — Ecuaciones 4, 5 y 6 | 3 |
| 10. Formalización: tripleta de perturbación — Ecuación 7                         | 4 |
| 11. Pipeline de tres etapas                                                      | 4 |
| 12. Etapa 1 — Generación de respuestas adversarias                               | 4 |
| 13. Etapa 2 — Extracción de caminos de relaciones                                | 5 |
| 14. Etapa 3 — Inserción de tripletas de perturbación                             | 5 |
| 15. Configuración de la réplica                                                  | 5 |
| 16. Ejecución con la API real de OpenAI                                          | 5 |

|                                                  |   |
|--------------------------------------------------|---|
| 17.Resultados reportados en el artículo original | 6 |
| 18.Desviaciones y limitaciones de alcance        | 6 |
| 19.Modelos de lenguaje evaluados en el artículo  | 6 |
| 20.Conclusiones                                  | 7 |
| 21.Referencias / Gracias                         | 7 |
| 22.Apéndice — Mapa de módulos                    | 7 |
| 23.Apéndice — Secuencia de ejecución             | 7 |
| Preguntas frecuentes anticipadas                 | 7 |

---

## Slide 1 Portada

**Tiempo sugerido: 0:20**

Saludo breve, nombre, y una frase de encuadre: “*Voy a presentar la reproducción de un ataque de envenenamiento de conocimiento contra sistemas KG-RAG, del artículo de Zhao et al. (2025).*” No hay contenido técnico en este slide; sirve solo para situar al público antes de la diapositiva 3, que es la más importante de toda la exposición.

## Slide 2 Contenido

**Tiempo sugerido: 0:20**

Recorrer el índice en una frase por sección, sin detenerse: motivación, marco teórico, método implementado, réplica del experimento, desviaciones y limitaciones, conclusiones. No profundizar en nada todavía — es solo un mapa.

## Slide 3 Nota sobre la disponibilidad de código

**Tiempo sugerido: 0:45**

Esta es la diapositiva más importante de la exposición y debe decirse con seguridad, no como disculpa. El punto central: el artículo de Zhao et al. (2025) no publica un repositorio de código —se verificó en el PDF, en el HTML de arXiv y en la página de resumen— así que, a diferencia de otros artículos de la lista del curso, aquí no es posible clonar y ejecutar el código original porque ese código no existe públicamente. Por eso todo lo que sigue es una **implementación propia**, construida directamente a partir de la descripción metodológica y de las ecuaciones 1 a 7 del artículo, no a partir de un código fuente ajeno. Cerrar remitiendo a que las desviaciones frente al método original están documentadas de forma explícita más adelante (diapositiva 18), para que la implementación sea auditable frente al texto original.

*Si el profesor pregunta “¿por qué no clonaste el repo?” aquí mismo:* repetir que se buscó activamente y no existe, y que se avisó de esto con anticipación (ver sección de preguntas frecuentes al final de este documento).

## Slide 4 ¿Por qué importa la seguridad de KG-RAG?

**Tiempo sugerido: 0:45**

Encadenar las cuatro viñetas como una idea continua, no como lista suelta: RAG dejar de depender solo de lo memorizado en el entrenamiento y apoyarse en una fuente externa; cuando esa fuente es un grafo de conocimiento (KG), el sistema puede razonar explícitamente, salto a salto, entre entidades — eso es KG-RAG, y es lo que lo hace interpretable. Pero la misma edición del grafo que permite mantenerlo actualizado es la puerta de entrada del atacante: una sola tripleta falsa, bien conectada, puede desviar una cadena de razonamiento completa sin alterar visiblemente el resto del grafo. Esa última frase es el gancho hacia el ejemplo de la siguiente diapositiva.

## Slide 5 Ejemplo motivador

**Tiempo sugerido: 1:00**

Leer la pregunta en voz alta, luego señalar la columna izquierda: ese es el camino de razonamiento legítimo hacia la respuesta correcta (Estados Unidos), tres saltos completos. Luego la columna derecha: con solo **una arista falsa** insertada (Manchester -cityOf->England), aprovechando una tripleta ya existente y verídica pero ajena a la película (England -containedIn->United Kingdom), se completa una ruta alternativa de dos saltos hacia una respuesta incorrecta. Enfatizar: la tripleta insertada por sí sola no basta, se apoya en algo que ya estaba en el grafo — así es como el ataque se mantiene sigiloso. Cerrar con el objetivo declarado: documentar una implementación propia de este mecanismo, de extremo a extremo, sobre un caso verificable a mano.

## Slide 6 Grafo de conocimiento y caminos de relaciones

**Tiempo sugerido: 0:45**

Tres definiciones que se van a usar sin volver a explicar durante el resto de la charla, así que conviene ir despacio aquí:

- **Grafo de conocimiento**  $G = (E, R, T)$ : un conjunto de tripletas  $(e_h, r, e_t)$ , cada una una arista dirigida y etiquetada de una entidad cabeza a una entidad cola.
- **Camino de razonamiento**  $p$ : una secuencia de saltos entidad-relación-entidad; su **camino de relaciones** asociado  $w = (r_1, \dots, r_l)$  es la misma secuencia pero descartando las entidades intermedias — solo importa el patrón de relaciones, no por dónde pasa exactamente.
- **Anclaje (*grounding*)**: la operación inversa. Dado un camino de relaciones  $w$  y una entidad de inicio, encontrar qué entidades reales del grafo se alcanzan al recorrerlo.

El anclaje es la pieza que se vuelve a usar literalmente en la Ecuación 7 (diapositiva 10) y en la Etapa 3 del ataque (diapositiva 14), así que vale la pena remarcarlo.

## Slide 7 Pipeline de un sistema KG-RAG

**Tiempo sugerido: 1:00**

Dos ecuaciones, una por etapa de un sistema KG-RAG normal (sin ataque todavía): **Ec. 1** ( $G_q = \text{Retriever}(q; G)$ ) es la etapa de recuperación: dado el grafo completo  $G$  y la pregunta  $q$ , se extrae un subgrafo  $G_q$  con solo la información relevante. **Ec. 2** ( $a = f_\theta(\text{Prompt}[q, G_q])$ ) es la etapa de generación: un LLM con parámetros  $\theta$  recibe la pregunta y el subgrafo recuperado combinados en un prompt, y produce la respuesta final  $a$ . La vulnerabilidad estructural que sostiene todo el artículo aparece justo aquí: si  $G$  contiene tripletas falsas bien conectadas, la Ec. 1 puede incluirlas en  $G_q$  sin ningún mecanismo que las distinga de las legítimas — el atacante no necesita tocar la Ec. 2 en absoluto.

## Slide 8 Modelo de amenaza (caja negra)

**Tiempo sugerido: 1:00**

La Ecuación 3 formaliza el objetivo del atacante:  $\hat{A} = \text{KG-RAG}(q; \hat{G})$ , con  $a^* \notin \hat{A}$ , sobre un grafo envenenado  $\hat{G} = \{E, R, T \cup \hat{T}\}$ . En palabras: el atacante inserta un conjunto de tripletas adversarias  $\hat{T}$  en el grafo original, de modo que cuando el sistema KG-RAG completo opera sobre el grafo envenenado  $\hat{G}$ , el conjunto de respuestas que produce  $\hat{A}$  **ya no contiene** la respuesta correcta original  $a^*$ . Tres restricciones importantes de remarcar en voz alta:

1. El atacante **no conoce** el recuperador, el LLM (ni su arquitectura ni sus parámetros), ni la respuesta correcta de la pregunta que está atacando — de ahí “caja negra”.
2. Su única capacidad es **insertar** tripletas; no puede borrar ni modificar las existentes.
3. Restricciones de sigilo: cada tripla insertada reutiliza entidades y relaciones *ya existentes* en el grafo (nunca inventa vocabulario nuevo), y el número de tripletas por respuesta adversaria está acotado por un presupuesto  $K$ .

## Slide 9 Formalización: objetivo de extracción de caminos (I–II) — Ecuaciones 4, 5 y 6

**Tiempo sugerido: 1:45**

Esta es la diapositiva que fusiona dos slides originales (Ec. 4–5 y Ec. 6) porque las tres ecuaciones son un solo argumento en tres pasos. Conviene narrarlas como una cadena, no como tres fórmulas sueltas:

**Ecuación 4 — el objetivo ideal.**  $\max_{\theta} \mathbb{E}_{w \sim W^*} [\log P_{\theta}(w | q)]$ . Se busca el conjunto de parámetros  $\theta$  que maximiza la probabilidad esperada, bajo la distribución de caminos “correctos”  $W^*$ , de que el modelo genere ese camino de relaciones  $w$  dada la pregunta  $q$ .  $W^*$  es el conjunto de caminos de relaciones **más cortos** entre la entidad tema y la respuesta correcta en el grafo — se le llama la *supervisión débil* del problema porque no hay etiquetas manuales de “el camino correcto es este”; se deriva algorítmicamente calculando caminos mínimos.

**Ecuación 5 — la versión computable.**  $\arg \max_{\theta} \frac{1}{|W^*|} \sum_{w \in W^*} \log P_{\theta}(w | q)$ . La Ec. 4 tiene una esperanza matemática que no se puede calcular directamente sin conocer la distribución completa de  $w$ . La Ec. 5 la resuelve asumiendo que  $w$  es uniforme dentro de  $W^*$ : bajo ese supuesto, la esperanza se aproxima simplemente **promediando** el log-verosímil sobre todos los caminos del conjunto finito  $W^*$ . Es literalmente la función de pérdida que se escribiría en código — el paso de “qué queremos” (Ec. 4) a “qué se puede entrenar” (Ec. 5).

**Ecuación 6 — la descomposición práctica.**  $\frac{1}{|W^*|} \sum_{w \in W^*} \log \prod_{i=1}^{|w|} P_{\theta}(r_i | r_{<i}, q)$ . Un camino de relaciones  $w = (r_1, \dots, r_l)$  es una **secuencia**, así que su probabilidad  $P_{\theta}(w | q)$  de la Ec. 5 se factoriza con la regla de la cadena en el producto de las probabilidades de cada relación individual  $r_i$ , condicionada a todas las relaciones generadas antes ( $r_{<i}$ ) y a la pregunta  $q$ . Esto convierte el problema en algo idéntico a modelado de lenguaje estándar, pero prediciendo *relaciones del grafo* en vez de palabras: entrenar el modelo para proponer buenos caminos equivale a entrenarlo para predecir, relación por relación, la siguiente más plausible.

Cerrar con el puente hacia las desviaciones: el artículo resuelve este objetivo **entrenando** un LLM específico de KG (LLM\_RoG, basado en LLaMA-2-7B) con estas tres ecuaciones como función de pérdida. Esta implementación logra el mismo efecto — caminos anclables en el grafo real — sin entrenar nada, restringiendo el vocabulario de un LLM de propósito general (detalle completo en la diapositiva 18).

## Slide 10 Formalización: tripleta de perturbación — Ecuación 7

**Tiempo sugerido: 1:00**

$E_{l-1} = \text{Grounding}(e_q, w'; G)$ , con  $w' = (r_1, \dots, r_{l-1})$ . Aquí se retoma el anclaje definido en la diapositiva 6.  $w'$  es el **prefijo** del camino de relaciones  $w$ : todas sus relaciones menos la última ( $r_l$ ). Se ancla ese prefijo desde la entidad tema  $e_q$ , recorriendo  $e_q \xrightarrow{r_1} \dots \xrightarrow{r_{l-1}} e_{l-1}$  sobre el grafo real, y se obtiene el conjunto  $E_{l-1}$  de entidades efectivamente alcanzadas. Por cada entidad alcanzada, se construye la tripleta de perturbación  $(e_{l-1}, r_l, \hat{a})$ : se adjunta la última relación del camino, apuntando directamente a la respuesta adversaria  $\hat{a}$ . En palabras simples: se recorre el camino real hasta el penúltimo paso, y el último salto se “clava” artificialmente hacia la respuesta que se quiere inducir. **Estrategia de respaldo**: si el prefijo no se puede anclar en el grafo (porque ninguna entidad real sigue ese patrón de relaciones), se sintetiza una cadena con entidades puente aleatorias que preserva el mismo patrón de relaciones hasta llegar a  $\hat{a}$  — así el ataque nunca se bloquea por falta de anclaje.

## Slide 11 Pipeline de tres etapas

[Recortable en vivo si falta tiempo]

**Tiempo sugerido: 0:40**

Diapositiva puramente de transición: mapea cada etapa del método a su módulo de código (`adversarial_answers.py`, `relation_paths.py`, `perturbation.py`) y a la función orquestadora `run_attack(kg, question, topic_entity, llm, n_answers=5, budget_k=4)`. Si el tiempo aprieta, se puede pasar directo a la Etapa 1 mencionando de paso que las tres etapas están encadenadas por `run_attack` con  $N = 5$  y  $K = 4$  como valores por defecto (los mismos del artículo).

## Slide 12 Etapa 1 — Generación de respuestas adversarias

**Tiempo sugerido: 1:00**

Mostrar el prompt (tomado literalmente del artículo): se le pide al LLM cinco nombres de entidad que sean respuestas *incorrectas* pero plausibles o confusas para la pregunta. El problema es que el LLM puede alucinar nombres que ni siquiera existen en el grafo actual — una respuesta adversaria inútil si la entidad no está ahí para anclar un camino hacia ella. Por eso se repiten rondas de generación y se aplica **coincidencia difusa** (`difflib.get_close_matches`) contra el vocabulario real de entidades del grafo, hasta reunir  $N$  respuestas que sí existen como nodos.

## Slide 13 Etapa 2 — Extracción de caminos de relaciones

**Tiempo sugerido: 1:00**

Aquí vive la desviación principal frente al artículo, aunque se detalla recién en la diapositiva 18 — en esta diapositiva basta con describir el mecanismo: se calcula el vocabulario de relaciones alcanzables en pocos saltos desde la entidad tema (`neighborhood_relations`), y se le pide al LLM caminos de relaciones usando **solo** ese vocabulario, con un prompt propio de esta implementación (el artículo resuelve esta etapa entrenando un modelo, no con un prompt). Cualquier camino que use una relación fuera del

vocabulario permitido se descarta en el post-procesamiento. La razón de que esto funcione: restringir el vocabulario garantiza caminos anclables en el grafo real, sin necesidad de entrenar un modelo específico de KG.

## Slide 14 Etapa 3 — Inserción de tripletas de perturbación

**Tiempo sugerido: 1:00**

Esta etapa es la aplicación directa de la Ecuación 7 (diapositiva 10): estrategia principal — ancla el prefijo del camino desde la entidad tema y adjunta la última relación hacia la respuesta adversaria; estrategia de respaldo — si el prefijo no ancla, muestrea entidades puente al azar preservando el mismo patrón de relaciones. Cerrar con las dos reglas operativas: presupuesto  $K$  tripletas por respuesta adversaria, y nunca se duplica una triplete que ya exista en el grafo (eso delataría el ataque).

## Slide 15 Configuración de la réplica

**Tiempo sugerido: 0:45**

Transición hacia la demo: instalación y variable de entorno, y los dos modos de ejecución del script. Explicar el caso de prueba en una frase: grafo construido a mano con 13 tripletas sobre *Manchester By The Sea*, atacando la pregunta de en qué país se filmó. Distinguir los dos modos: `-offline` usa `StaticLLMClient` con respuestas fijas — siempre inserta las mismas 2 tripletas hacia *United Kingdom*, es la referencia reproducible del proyecto; sin el flag, se llama a la API real de `gpt-4o-mini` a temperatura 0.7, sin respuestas fijas, así que cada corrida puede variar — eso se muestra en la siguiente diapositiva.

## Slide 16 Ejecución con la API real de OpenAI

**Tiempo sugerido: 1:45**

**Esta diapositiva fusiona la ejecución real con su interpretación.** Mostrar primero la salida del script: con el modelo real, la respuesta adversaria que sobrevivió la coincidencia difusa fue *Florida*, no *United Kingdom* como en el modo offline — porque el modelo real no es determinista y propone un conjunto distinto de países en cada corrida. Explicar por qué *Florida* sobrevivió: existe en este grafo solo por dos tripletas ajenas a la película (`Miami, locatedIn, Florida` y `Florida, stateOf, United States`), así que la coincidencia es accidental, del mismo tipo que la de *United Kingdom* en el modo offline. De los dos caminos de relaciones propuestos, señalar que (`filmPlace, starring`) es **anclable pero semánticamente incoherente**: adjuntar `starring` (protagonizada por) a una ubicación como Manchester no tiene sentido real, aunque la restricción de vocabulario de la Etapa 2 lo admite sin problema — es un límite honesto de esa desviación. Cerrar con la interpretación: con solo **2 tripletas insertadas** sobre 13 originales se crean dos rutas de razonamiento alternativas hacia *Florida*, sin ninguna señal estructural que las distinga de la ruta legítima hacia *United States* — esto confirma el mecanismo central del artículo, incluso con un camino semánticamente extraño. Y el límite de reproducibilidad: sin semilla fija para el muestreo del LLM, esta corrida específica no es reproducible byte a byte (por eso el modo `-offline` es la referencia canónica del proyecto).

## Slide 17 Resultados reportados en el artículo original

**Tiempo sugerido: 1:15**

Aclarar primero que esta tabla **no es de esta implementación**: son los resultados del artículo (Tabla 3, WebQSP) que se muestran como contexto de escala, ya que el caso de Manchester By The Sea no reproduce estas cifras. Con  $N = 5$  respuestas adversarias y  $K = 4$  tripletas cada una (máximo 20

tripletas por pregunta), los cuatro sistemas KG-RAG evaluados sufren caídas sustanciales de F1 frente a una línea base de modificación aleatoria (que apenas los afecta). El hallazgo clave del artículo, y el que conecta con la Etapa 2 de este proyecto: la vulnerabilidad principal está en la etapa de **recuperación** (más del 87% de las tripletas adversarias son recuperadas); la etapa de generación con el LLM es más resistente, pero exhibe una respuesta polarizada — o rechaza el contenido adversario por completo, o lo adopta predominantemente, sin término medio.

## Slide 18 Desviaciones y limitaciones de alcance

**Tiempo sugerido: 1:30**

Esta diapositiva fusiona “Desviaciones frente al método original” con “Limitaciones de alcance”. Primero la única desviación deliberada: el artículo entrena LLM\_RoG (LLaMA-2-7B) con el objetivo de las Ecuaciones 4–6 (diapositiva 9); esta implementación logra caminos anclables restringiendo el vocabulario de relaciones de un LLM de propósito general al vecindario real de la entidad tema, sin GPU ni entrenamiento — recalcar que el resto del pipeline (respuestas adversarias por coincidencia difusa, inserción con estrategia de respaldo) sigue el artículo directamente, sin modificaciones de fondo. Luego las tres limitaciones de alcance, dichas con transparencia y sin sonar como excusa:

1. No se integra ningún sistema KG-RAG objetivo (RoG, GCR, G-retriever, SubgraphRAG): la validación se detiene en el grafo envenenado, verificado por inspección directa, sin medir métricas de respuesta final.
2. Sin benchmarks a gran escala: 13 tripletas construidas a mano, frente a los miles de preguntas y subgrafos de WebQSP/CWQ.
3. Modelo de lenguaje distinto (**gpt-4o-mini** en vez de GPT-4) — remitir a la siguiente diapositiva para el detalle completo.

Cerrar con el trabajo futuro: integrar un recuperador y generador mínimos para medir el efecto sobre la respuesta final, y ejecutar sobre un subconjunto real de WebQSP/CWQ.

## Slide 19 Modelos de lenguaje evaluados en el artículo

**Tiempo sugerido: 1:15**

Esta es la Tabla 5 del artículo: robustez del método RoG en WebQSP variando **solo** el LLM de la etapa de generación, con recuperación y prompts fijos. Recorrer la tabla de forma rápida por categoría, no fila por fila: modelos open-source (Llama-2-7B-chat-hf, Llama-3.1-8B-Instruct) parten más bajo y caen más en términos absolutos; los closed-source (GPT-3.5-turbo, GPT-4) y el modelo MoE (DeepSeek-V3-0324) rinden mejor tanto limpio como atacado; y el modelo específico de KG (LLM\_RoG) logra el mejor desempeño limpio pero también sufre la **mayor** degradación relativa bajo ataque (46 %) — hallazgo contraintuitivo del artículo: saber más sobre la estructura del grafo también lo hace más vulnerable a su manipulación. Aclarar que para generar las respuestas adversarias del propio ataque el artículo usa GPT-4, y que GCR/SubgraphRAG usan GPT-3.5-turbo como backend de evaluación — ninguno de estos seis modelos coincide con el usado en esta implementación. Cerrar con el **alertblock**: esta implementación usa únicamente **gpt-4o-mini** en las dos etapas basadas en LLM (respuestas adversarias y caminos de relaciones). **Si preguntan por qué**, la respuesta completa está en la sección de preguntas frecuentes al final de este documento.

## Slide 20 Conclusiones

### Tiempo sugerido: 0:45

Cuatro puntos, dichos con seguridad y sin sobre-explicar (ya se dijo todo antes): se reprodujo el pipeline de tres etapas de forma ejecutable y verificable, con demostración de extremo a extremo contra la API real; el caso replicado confirma el mecanismo central del artículo (una perturbación mínima e indistinguible estructuralmente puede desviar una cadena de inferencia completa); la simplificación principal — vocabulario restringido en vez de modelo entrenado — preserva la propiedad esencial (caminos anclables) pero no garantiza coherencia semántica, como mostró el camino **starring** sobre una ubicación en la ejecución real; y el paso natural siguiente es integrar un sistema KG-RAG objetivo para medir el impacto sobre la respuesta final.

## Slide 21 Referencias / Gracias

### Tiempo sugerido: 0:20

Leer la referencia en formato APA y agradecer. No hay contenido nuevo que explicar.

## Slide 22 Apéndice — Mapa de módulos

### [Material de respaldo — mostrar solo si preguntan]

Solo mostrar si preguntan “¿dónde vive cada pieza del código?”. Frase de resumen lista para usar: *“Las tres etapas del método están en `adversarial_answers.py`, `relation_paths.py` y `perturbation.py`; `kg.py` y `llm.py` son utilidades compartidas que las tres usan; `attack.py` las encadena; `demo_manchester.py` es el único punto de entrada ejecutable desde la terminal.”*

## Slide 23 Apéndice — Secuencia de ejecución

### [Material de respaldo — mostrar solo si preguntan]

Solo mostrar si preguntan “¿cómo se ejecuta exactamente el script paso a paso?”. Seguir la numeración del slide: construcción del grafo, instanciación del LLM (real u offline), llamada a `run_attack` (que internamente llama a las tres funciones de las tres etapas en orden), y finalmente el resumen impreso comparando `out_edges` antes y después del ataque.

## Preguntas frecuentes anticipadas

### ¿Por qué no usaste GPT-4 o GPT-3.5-turbo, como el artículo?

Cuatro capas de respuesta, de la más fuerte a la más técnica:

1. **Vigencia del modelo.** OpenAI retira GPT-3.5-turbo el 23 de octubre de 2026 y GPT-4 está en esa misma ola de deprecaciones de 2026. Construir el proyecto sobre modelos que la propia OpenAI está apagando durante su desarrollo no es sostenible; `gpt-4o-mini` es el modelo económico vigente y activamente soportado.
2. **Costo y volumen de llamadas.** El pipeline hace múltiples rondas de generación por etapa (hasta  $N = 5$  candidatos con reintentos de coincidencia difusa) y se ejecuta repetidamente durante desarrollo y pruebas — un modelo “mini” es proporcional a ese volumen.
3. **La tarea es acotada, no el rol completo evaluado en el artículo.** Aquí el LLM no es el generador final de un sistema KG-RAG completo (eso está fuera de alcance); cumple dos tareas auxiliares más simples: listas cortas de nombres de entidad y caminos de relaciones ya restringidos a un vocabulario filtrado.



4. **Ya está declarado como limitación**, no oculto: no se afirma que `gpt-4o-mini` iguale a GPT-4; se reconoce que puede producir candidatos de menor diversidad (diapositivas 18 y 19), pero el mecanismo demostrado es una operación estructural sobre el grafo, verificable por inspección directa independientemente del modelo que generó los candidatos de entrada.

### **¿Por qué no clonaste y ejecutaste el código del artículo, como pide el formato del curso?**

Porque el artículo no publica código: se verificó explícitamente en el PDF, en el HTML de arXiv y en la página de resumen de arXiv, y no hay repositorio ni enlace a implementación oficial en ninguno de los tres lugares. Ante la ausencia de código que clonar, la única opción compatible con el objetivo del curso (reproducir el método) es una implementación propia construida directamente desde la descripción metodológica y las ecuaciones del artículo — que es exactamente lo que documenta este trabajo, con sus desviaciones declaradas de forma explícita en vez de ocultas.

### **¿Esto reproduce las cifras de degradación del artículo (Tablas 3, 4 y 5)?**

No, y eso está delimitado desde el inicio como parte del alcance: la validación se detiene en el grafo envenenado, verificado por inspección directa de las tripletas insertadas, sin integrar ningún sistema KG-RAG objetivo que produzca una respuesta final medible con esas métricas. Las tablas del artículo que aparecen en la presentación (diapositivas 17 y 19) se muestran como contexto de escala y como comparación de modelos, no como resultados propios.